
Informe de despliegue

Proyecto: Autogestión-Sena

Tabla de Contenidos

1. Introducción
2. Arquitectura del Sistema
3. Requisitos Técnicos
4. Instalación y Configuración
5. Estructura del Proyecto
6. Compilación y Despliegue
7. Referencias y Contacto

1. Introducción

1.1 Propósito del Documento

El presente informe describe el proceso completo de despliegue del sistema Autogestión SENA, una solución tecnológica desarrollada para optimizar la gestión académica, administrativa y operativa de los aprendices, instructores, coordinadores y personal institucional del SENA.

El sistema está compuesto por cuatro componentes principales:

- Backend desarrollado con Django + Django REST Framework, encargado de la lógica del negocio, la seguridad, la autenticación y la comunicación con la base de datos.
- Frontend Web, construido con React + TypeScript, que provee una interfaz moderna, responsiva y de fácil uso para los diferentes roles del sistema.
- Aplicación móvil (APK) desarrollada en MAUI .NET, orientada a la presentación y consulta rápida de información para aprendices e instructores.
- Base de Datos MySQL, encargada del almacenamiento seguro, estructurado y escalable de toda la información institucional.

El documento presenta los requisitos técnicos, el proceso de instalación, la configuración de cada uno de los componentes y el procedimiento seguido para realizar el despliegue final en producción. Además, se incluyen detalles sobre la arquitectura utilizada, la estructura del proyecto y las herramientas empleadas para garantizar un despliegue seguro, eficiente y mantenible.

El objetivo de este informe es servir como guía oficial de despliegue, de modo que cualquier integrante del equipo técnico, actual o futuro, pueda replicar el proceso, mantener la infraestructura y realizar actualizaciones sin dificultad.

1.2 Audiencia

Este documento está dirigido a:

- Desarrolladores frontend y backend
- Arquitectos de software
- Administradores de sistemas
- Personal de QA y testing
- Equipos de soporte técnico

1.3 Stack Tecnológico

- **Frontend**

Tecnología	Versión	Propósito
React	18.3.1	Librería de JavaScript utilizada para construir interfaces de usuario dinámicas y basadas en componentes reutilizables. Facilita la gestión del estado y la navegación entre vistas.
TypeScript	5.8.3	Superset de JavaScript que añade tipado estático, mejorando la mantenibilidad, escalabilidad y detección temprana de errores en el código.
Tailwind CSS	3.4.17	Framework CSS utilitario que permite diseñar interfaces modernas, responsivas y consistentes de forma rápida mediante clases predefinidas.
Vite	5.4.19	Herramienta de desarrollo rápida utilizada para compilar, ejecutar y optimizar el proyecto React con TypeScript. Mejora los tiempos de carga y compilación.
Node js	20.19.0	Entorno de ejecución que permite gestionar dependencias del proyecto, ejecutar scripts de compilación y levantar el entorno local de desarrollo.
Fetch API	—	Se emplea para la comunicación entre el frontend y el backend mediante peticiones HTTP hacia los servicios REST.

LocalStorage / SessionStorage	—	Se utiliza para almacenar de manera local la sesión del usuario, roles, permisos y otros datos temporales.
Json	—	Formato estándar de intercambio de datos entre el frontend y el backend, facilitando la serialización de objetos y respuestas de API.

- **Backend**

Tecnología	Versión	Propósito
Python	3.10	Lenguaje principal para el backend y scripts de gestión.
Django	4.2.23	Framework web principal, ORM, administración y creación de APIs REST.
Django Rest Framework	3.16.0	Construcción y manejo de APIs RESTful.
MySQL	2.2.7	Base de datos relacional para la persistencia de datos.
Git	2.49.0	Control de versiones del código fuente.
Celery	5.5.3	Ejecución de tareas asíncronas y programación de trabajos.
Redis	5.2.1	Servicio externo como <i>broker</i> y <i>backend</i> para tareas asíncronas de Celery.
Swagger / drf-yasg	1.21.10	Generación automática de documentación Swagger/OpenAPI para las APIs.
pytest / unittest	latest	Frameworks para pruebas automáticas y validación del código.

- **Frontend móvil**

Tecnología	Versión	Propósito
.NET MAUI	8.0	Framework para crear la aplicación móvil multiplataforma (Android, iOS, Windows).
C#	12	Lenguaje principal para escribir la lógica de la aplicación y la interacción con la interfaz.
XAML	—	Lenguaje declarativo para construir la interfaz de usuario de la aplicación.

- **Base de datos:**

Se usa MySQL como sistema de gestión de bases de datos relacional (RDBMS). Claves foráneas, apropiado para datos estructurados como usuarios, asignaciones y registros históricos. Las credenciales y conexión se cargan desde el archivo `.env` (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD).

- **API REST basada en Django REST Framework (DRF)**

Para la API se usa Django REST Framework — DRF — con ViewSets y routers; autenticación JWT mediante `django-rest-framework-simplejwt` disponible en POST `/api/token/` y POST `/api/token/refresh/`; documentación OpenAPI generada con `drf-yasg` accesible en GET `/swagger/` y GET `/redoc/`; los endpoints se exponen bajo el prefijo `/api/` y están organizados por módulos, p. ej. `apps.security`, `apps.general`, `apps.assign`.

2. Arquitectura del Sistema

2.1 Arquitectura General

Fronted

El frontend del sistema se encuentra desarrollado bajo una arquitectura modular y organizada por funcionalidades (feature-based), lo que permite una alta mantenibilidad, escalabilidad y separación clara de responsabilidades. La aplicación está estructurada siguiendo buenas prácticas de diseño, asegurando un flujo ordenado entre componentes, vistas, rutas y servicios.

La arquitectura del frontend opera bajo los siguientes principios:

1. Separación entre vista, lógica y comunicación

- Las vistas (*pages*) usan componentes reutilizables (*components*).
- La lógica está encapsulada en hooks personalizados (*hook*).
- La comunicación con el backend se gestiona desde servicios centralizados (*Api/Services*).

2. Estructura modular por funcionalidades

Cada módulo del sistema tiene sus propios componentes, páginas y lógica, organizados de manera independiente pero coherente.

3. Gestión de rutas con protecciones

Las rutas privadas pasan por **ProtectedRoute** y **ProtectedLayout**, permitiendo el acceso solo a usuarios autenticados.

4. Encapsulación del estado del servidor

El estado proveniente del backend se maneja mediante consultas y actualizaciones centralizadas, integrándose de forma limpia con los componentes.

Backend

El sistema adopta una arquitectura N-Capas basada en módulos (N-Layer Architecture by Module), donde cada módulo del dominio (app) encapsula sus propias capas de entidad, repositorio, servicio y vista.

Esto promueve la separación de responsabilidades, alta cohesión, bajo acoplamiento y facilidad de mantenimiento, permitiendo escalar el sistema de manera ordenada y reutilizar componentes.

Móvil

Arquitectura del Proyecto: Model-View-ViewModel (MVVM)

El proyecto de la aplicación móvil de autogestión del SENA se ha desarrollado siguiendo el patrón de arquitectura de software Model-View-ViewModel (MVVM). Esta elección se alinea con las mejores prácticas para el desarrollo de aplicaciones con .NET MAUI, ya que promueve una clara separación de responsabilidades, facilita la mantenibilidad y mejora la capacidad de realizar pruebas unitarias.

La arquitectura MVVM divide la aplicación en tres componentes principales:

1. Model (Modelo)

El Modelo representa la capa de datos y la lógica de negocio de la aplicación. Es responsable de la gestión de los datos, incluyendo la comunicación con servicios externos o APIs. En este proyecto, los componentes del Modelo se encuentran principalmente en el directorio

Dtos: Contiene los Objetos de Transferencia de Datos (DTOs), que son clases simples que definen la estructura de los datos intercambiados con el backend (ej. ApprenticeDto.cs, UserDto.cs).

Services: Alberga los servicios que implementan la lógica para interactuar con la API, como realizar peticiones HTTP para obtener o enviar datos (ej. [UserService.cs](#)).

2. View (Vista)

La Vista es la capa de presentación, responsable de la interfaz de usuario (UI) y la experiencia visual del usuario. Se define mediante archivos XAML, que describen la estructura y el diseño de las pantallas de forma declarativa. Las Vistas se encuentran en el directorio Views:

Views: Contiene todas las páginas de la aplicación, como LoginPage.xaml, RegisterPage.xaml y HomePage.xaml. El código subyacente de estas vistas (archivos .xaml.cs) se mantiene al mínimo, delegando toda la lógica al ViewModel.

ContentViews: Incluye componentes de interfaz de usuario reutilizables, como BottomMenu.xaml, que pueden ser incrustados en múltiples vistas.

3. ViewModel (Modelo de Vista)

El ViewModel actúa como un puente entre la Vista y el Modelo. Expone los datos del Modelo a la Vista a través de propiedades enlazables (data binding) y maneja las acciones del usuario a través de comandos. Esto permite que la lógica de la interfaz de usuario se desarrolle y se pruebe de forma independiente a la interfaz gráfica. Los ViewModels se localizan en el directorio ViewModels:

ViewModels: Contiene las clases que encapsulan la lógica de presentación y el estado de cada vista, como LoginViewModel.cs.

La adopción de la arquitectura MVVM en este proyecto garantiza un código más limpio, modular y escalable, desacoplando la interfaz de usuario de la lógica de negocio y facilitando el trabajo colaborativo y el mantenimiento a largo plazo.

3. Requisitos Técnicos

3.1 Requisitos de Desarrollo

Software Requerido Backend

Componente	Versión Mínima	Propósito
Python	3.10 o superior	Lenguaje principal de desarrollo del backend y ejecución de scripts.
Django	4.2.23	Framework web principal para la construcción del backend y APIs REST.
MySQL / MariaDB	8.x / 10.x	Sistema de gestión de base de datos relacional utilizado para la persistencia de datos.
Redis	6.x o superior	Sistema de almacenamiento en memoria usado como <i>broker</i> y <i>backend</i> de tareas asíncronas.
Celery / django-celery-beat	5.x / 2.x	Ejecución y programación de tareas en segundo plano.
pip	Última versión estable	Gestor de paquetes para la instalación de dependencias de Python.
Git	2.49.0 (última estable)	Control de versiones y clonación del repositorio.
requirements.txt	—	Lista de dependencias necesarias para ejecutar el proyecto (instalación con pip install -r requirements.txt).

Software Requerido Frontend

Componente	Versión Mínima	Propósito
TypeScript	5.8.3	Compilación y chequeo de tipos para los archivos .ts/.tsx.

Tailwind CSS	3.4.17	Procesamiento de estilos y utilidades CSS.
Vite	5.4.19	Dev server y bundler rápido para React + TypeScript.
Node js	20.19.0	Ejecutar el entorno de desarrollo (Vite), instalar dependencias (npm/pnpm/bun) y compilar la app TypeScript/React.
Git	2.20	Clonar el repositorio, manejar ramas y commits.
Navegador moderno	Chrome/Edge/Firefox (última versión)	Probar la aplicación en dev/preview.
npm	9.x (incluido con Node 18+)	Gestor de paquetes para instalar dependencias definidas en package.json y ejecutar scripts (dev/build).

Software Requerido Móvil

Componente	Versión Mínima	Propósito
.NET SDK	8.0	Proporciona las bibliotecas y el tiempo de ejecución necesarios para compilar y ejecutar la aplicación .NET MAUI.
Visual Studio 2022	17.8	Entorno de Desarrollo Integrado (IDE) para escribir, depurar y desplegar el código de la aplicación.
Carga de trabajo .NET MAUI	Incluida con VS 2022	Instala todas las herramientas y SDKs específicos para el desarrollo de aplicaciones multiplataforma con .NET MAUI.

Sistema Operativo

- Windows: 10/11 (64-bit)
- Linux: Ubuntu 20.04+ o equivalente

3.2 Requisitos de Hardware

Mínimos

- Procesador: Intel Core i3 o equivalente
- RAM: 8 GB
- Almacenamiento: 10 GB disponibles
- Conexión a Internet

Recomendados:

- Procesador: Intel Core i5/i7 o equivalente
- RAM: 16 GB
- Almacenamiento: SSD con 20 GB disponibles
- Conexión a Internet estable

3.3 Dispositivos Móviles Compatibles

Android

- Versión mínima: Android 6.0 (API 23)
- Versión recomendada: Android 10+ (API 29+)
- Resoluciones soportadas: 320x480 hasta 1440x3040

4. Instalación y Configuración

4.1 Backend

Instalación y Configuración del Proyecto Backend

Para ejecutar el proyecto Backend de Autogestión Sena, se requiere un entorno con Python 3.10 o superior y las dependencias definidas en el archivo **requirements.txt**.

A continuación, se describen los pasos necesarios para la instalación y configuración del entorno de desarrollo:

1. Clonación del proyecto

Descargar el código fuente desde el repositorio Git correspondiente utilizando el siguiente comando:

```
git clone
```

```
<https://github.com/July173/Back-end-web-API-autogestionSena.git>
```

Luego, ingresar al directorio raíz del proyecto:

```
cd BACK-ENDPROYECTOFINAL2025
```

2. Creación del entorno virtual (*opcional pero recomendado*)

Se recomienda crear un entorno virtual para aislar las dependencias del proyecto:

```
python -m venv venv
```

Activar el entorno:

- En Windows:
venv\Scripts\activate
- En Linux/Mac:
source venv/bin/activate

3. Instalación de dependencias

Instalar todas las librerías y módulos requeridos ejecutando:

```
pip install -r requirements.txt
```

Esto instalará Django, Django REST Framework, Celery, Redis, y las demás dependencias necesarias para la ejecución del sistema.

4. Configuración de base de datos

Configurar las variables de entorno para MySQL o editar directamente el archivo:

```
core/settings.py
```

Asegurando que las credenciales de conexión correspondan a la base de datos configurada en el entorno local o de producción.

5. Migraciones de base de datos

Generar e implementar las migraciones correspondientes a los modelos del proyecto:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

6. Carga de datos iniciales

Actualizar la base de datos con la estructura y datos iniciales incluidos en el archivo **sql.sql** ubicado en la raíz del proyecto:

```
mysql -u <usuario> -p <nombre_base_datos> < sql.sql
```

7. Creación de usuario administrador (*opcional*)

Este paso es opcional, pero se recomienda para acceder al panel administrativo de Django:

```
python manage.py createsuperuser
```

8. Ejecución del servidor de desarrollo

Iniciar el servidor local de Django:

python manage.py runserver

Acceder desde el navegador a la dirección:

http://127.0.0.1:8000/

9. Ejecución de tareas asíncronas (Celery)

Para permitir el funcionamiento de las tareas en segundo plano, ejecutar los siguientes comandos en terminales separadas:

celery -A core worker --loglevel=info

celery -A core beat --loglevel=info

10. Acceso a la documentación de la API

El proyecto cuenta con documentación automática generada con drf-yasg (Swagger).-

Una vez iniciado el servidor, puede consultarse en el navegador:

http://127.0.0.1:8000/swagger/

4.2 Frontend

Esta sección describe el proceso necesario para instalar, configurar y ejecutar el frontend del sistema en un entorno local de desarrollo.

1. Clonar el repositorio

Abre PowerShell.

Ejecuta el comando:

git clone

<https://github.com/July173/Front-end-web-Autogestion-Sena.git>

Entra al proyecto:

cd <https://github.com/July173/Front-end-web-Autogestion-Sena.git>

2. Verificar versiones instaladas

Antes de instalar el proyecto, verifica que tu sistema tenga Git, Node.js, npm o el gestor que utilices.

Ejecuta en PowerShell:

git --version

node --version

npm --version

3. Instalar dependencias

Usando npm (recomendado)

npm install

Alternativa con pnpm

pnpm install

O con Bun (opcional)

bun install

4. Configuración de variables de entorno

El proyecto utiliza una variable principal de configuración:

VITE_API_BASE_URL

Esta variable es leída en ConfigApi.ts, y define la URL base desde donde el frontend consume las APIs.

4.1 Crear un archivo .env

En la raíz del proyecto frontend, crea el archivo:

.env

Ejemplo de contenido:

VITE_API_BASE_URL=<http://localhost:8000/api/>

Notas importantes:

Las variables de entorno deben empezar con VITE_ para que Vite las cargue.

El archivo .env debe ubicarse en la raíz del frontend, al mismo nivel que package.json.

5. Ejecutar la aplicación en modo desarrollo

Para iniciar el servidor de desarrollo con Vite:

npm run dev

o con otros gestores:

pnpm dev

bun dev

Esto generará una URL local, usualmente:

<http://localhost:5173/>

Ábrela en tu navegador para visualizar la aplicación.

6. Ejecutar build de producción

Para generar la versión optimizada para despliegue:

npm run build

Después puedes previsualizarla:

npm run preview

El resultado final quedará en la carpeta:

/dist

4.3 Móvil

Guía de Configuración e Instalación del Proyecto Móvil

Esta guía describe los pasos necesarios para configurar el entorno de desarrollo, compilar e instalar la aplicación móvil de autogestión del SENA en un emulador o dispositivo Android.

1. Configuración del Entorno de Desarrollo

Para poder compilar y trabajar en el proyecto, es necesario instalar el siguiente software:

Paso 1: Instalar Visual Studio 2022

Descargue e instale Visual Studio 2022 (versión 17.8 o superior) desde el sitio web oficial de Microsoft.

Durante la instalación, en la pestaña "Cargas de trabajo", asegúrese de seleccionar la carga de trabajo "Desarrollo de la interfaz de usuario de aplicaciones multiplataforma .NET" (o ".NET Multi-platform App UI development").

Esta selección instalará automáticamente todos los componentes necesarios, incluyendo el SDK de .NET 8, las plantillas de .NET MAUI y el SDK de Android.

Paso 2: Clonar el Repositorio

git clone

<https://github.com/July173/Front-end-Mobile-Autogestion-Sena.git>

Instale un cliente de Git si aún no lo tiene.

Abra una terminal o consola de comandos y clone el repositorio del proyecto en su máquina local

Paso 3: Configurar un Emulador de Android (Opcional)

Si no dispone de un dispositivo Android físico para las pruebas, puede crear un emulador:

En Visual Studio, vaya al menú Herramientas > Android > Administrador de dispositivos Android.

Haga clic en "+ Nuevo" para crear un nuevo emulador. Se recomienda seleccionar una imagen de sistema reciente (ej. API 33 o superior) para garantizar la compatibilidad.

Inicie el emulador desde el administrador de dispositivos para asegurarse de que funciona correctamente.

2. Compilación e Instalación de la Aplicación

Una vez configurado el entorno, siga estos pasos para instalar la aplicación:

Paso 1: Abrir el Proyecto

Navegue a la carpeta donde clonó el repositorio.

Abra el archivo de solución AutogestionSenaMaui.sln con Visual Studio 2022.

Paso 2: Restaurar Dependencias

Visual Studio debería restaurar automáticamente los paquetes NuGet necesarios al abrir el proyecto. Si esto no ocurre, haga clic derecho sobre la solución en el "Explorador de soluciones" y seleccione "Restaurar paquetes NuGet".

Paso 3: Seleccionar el Dispositivo de Destino

En la barra de herramientas principal de Visual Studio, asegúrese de que el proyecto de inicio sea AutogestionSenaMaui.

En el menú desplegable de dispositivos, seleccione el destino donde desea instalar la aplicación. Puede ser:

Un emulador de Android que haya configurado previamente.

Un dispositivo Android físico conectado a su computadora (asegúrese de que tenga habilitado el modo de desarrollador y la depuración por USB).

Paso 4: Ejecutar e Instalar

Haga clic en el botón de inicio verde (▶) o presione la tecla F5.

Visual Studio compila el proyecto, generará el paquete de la aplicación (APK) y lo instalará automáticamente en el dispositivo de destino seleccionado.

Una vez finalizado el proceso, la aplicación se iniciará para su uso y depuración.

5. Estructura del Proyecto

5.1 Árbol de Directorios

Fronted:

https://drive.google.com/file/d/14y2pP9Pt1LgwEj8QESz0WWGhfqYziigv/view?usp=drive_link

Backend:

https://drive.google.com/file/d/1qedVp_LWIkxIO0DNUO-XCXKPdF5azNNj/view?usp=drive_link

Móvil:

https://drive.google.com/file/d/1CBFQVyaOdLxoZ3OJTQxXy_Y9CHvsZRG/view?usp=drive_link

6. Compilación y Despliegue

6.1 Entornos de Despliegue

maneja cuatro entornos:

Entorno	Propósito	API URL
Dev	Desarrollo local del programador. Aquí se agregan nuevas funciones y se realizan pruebas unitarias.	http://167.114.98.199/swagger/
qa	Ambiente de pruebas internas donde QA valida flujos, módulos y correcciones antes de staging.	(No disponible actualmente)
staging	Réplica previa a producción usada para pruebas finales, cargas de datos, validación de rendimiento.	(No disponible actualmente)
producción/main	Entorno en uso real por aprendices, empresas y talento humano. Es la versión estable del sistema.	http://167.114.98.199/

6.2 Compilación APK de Pruebas

Esta sección documenta el proceso para generar el archivo APK (Android Package Kit) de la aplicación que se utilizará para realizar pruebas funcionales.

1. Entorno y Herramientas

Componente	Versión/Especificación	Propósito
IDE Principal	Visual Studio 2022 (v17.8+)	Desarrollo y compilación
IDE Alternativo	Visual Studio Code + extensión C#	Edición de código

Framework	.NET MAUI (.NET 8.0)	Framework multiplataforma
.NET SDK	8.0.x	Compilación y ejecución
Android SDK	API 21+ (mínimo Android 5.0)	Plataforma objetivo
Android Build-Tools	34.0.0 Herramientas de compilación	34.0.0 Herramientas de compilación
Java JDK	11 o superior	Requerido por Android SDK

2. Procedimiento de Compilación - Visual Studio 2022

Opción A: Compilación desde la Interfaz Gráfica

Abrir la solución:

- Doble clic en AutogestionSenaMaui.sln
- Esperar a que se carguen todos los proyectos
- Seleccionar configuración de Build:

En la barra de herramientas superior:

- Configuración: Seleccionar Debug
- Plataforma: Seleccionar Any CPU
- Framework: Seleccionar net8.0-android

Compilar la solución:

- Menú: Build > Build Solution (o presionar Ctrl+Shift+B)
- Esperar a que finalice la compilación
- Verificar en la ventana de salida: Build succeeded

Generar APK:

- Menú: Build > Publish Selection
- O hacer clic derecho en el proyecto > Publish

Opción B: Compilación desde Terminal (PowerShell) - RECOMENDADO

- 1. Navegar al directorio del proyecto
cd "C:\Users\braya\Desktop\Front-end-Mobile-Autogestion-Sena"
- 2. Limpiar compilaciones anteriores (evita conflictos)
dotnet clean -f net8.0-android -c Debug
- 3. Restaurar paquetes NuGet
dotnet restore
- 4. Compilar APK de Debug (Pruebas)
dotnet build -f net8.0-android -c Debug
- 5. Verificar que la APK se generó correctamente
Get-ChildItem -Path "bin\Debug\net8.0-android\" -Filter "*.apk"
-Recurse | Format-Table Name, Length, LastWriteTime

3. Ubicación del Archivo APK Generado

Después de la compilación exitosa, el archivo APK se encuentra en:

```
Front-end-Mobile-Autogestion-Sena/
├── bin/
│   ├── Debug/
│   │   ├── net8.0-android/
│   │   │   ├── com.companyname.autogestionsena.maui.apk      (APK sin
│   │   │   │   firmar)
│   │   │   └── com.companyname.autogestionsena.maui-Signed.apk (APK
│   │   │   │   firmada )
```

Ruta completa:

```
C:\Users\braya\Desktop\Front-end-Mobile-Autogestion-Sena\bin\Debug\net8.
0-android\com.companyname.autogestionsena.maui-Signed.apk
```

4. Nomenclatura y Versionamiento de APK

Para mantener un control adecuado de las versiones de prueba, se recomienda seguir esta convención de nomenclatura:

AutoGestion-SENA-[version]-[fecha]-[entorno].apk

Ejemplos:

- AutoGestion-SENA-1.0.0-2025.11.27-debug.apk
- AutoGestion-SENA-1.0.0-2025.11.27-qa.apk

Script para renombrar APK automáticamente:

```
# Variables
$version = "1.0.0"
$fecha = Get-Date -Format "yyyy.MM.dd"
$entorno = "debug"
$origen =
"bin\Debug\net8.0-android\com.companyname.autogestionsena.maui-
Signed.apk"
$destino =
"releases\AutoGestion-SENA-$version-$fecha-$entorno.apk"

# Crear carpeta releases si no existe
New-Item -ItemType Directory -Path "releases" -Force | Out-Null

# Copiar y renombrar
Copy-Item $origen $destino
Write-Host " APK generada: $destino" -ForegroundColor Green
```

5. Características de la APK de Pruebas (Debug)

Característica	APK Debug (Pruebas)	APK Release (Producción)
Optimización de código	Deshabilitada	Habilitada
Símbolos de depuración	Incluidos	Removidos
Logging/Debug.WriteLine	Activo	Limitado
Debuggeable	Sí	No
Tamaño	~80-120 MB	~40-60 MB

aproximado		
Rendimiento	Más lenta	Más rápida
Firma	Debug keystore	Keystore de producción
Uso recomendado	Testing interno, QA	Google Play, distribución

6. Verificar Integridad de la APK

Ver información de la APK generada

\$apk = Get-Item

"bin\Debug\net8.0-android\com.companyname.autogestionsena.maui-Signed.apk"

Write-Host "Información de la APK:" -ForegroundColor Cyan

Write-Host " Nombre: \$(\$apk.Name)"

Write-Host " Tamaño: \$([math]::Round(\$apk.Length / 1MB, 2)) MB"

Write-Host " Fecha de creación: \$(\$apk.CreationTime)"

Write-Host " Ruta: \$(\$apk.FullName)"

Verificar firma (requiere Java/keytool)

jarsigner -verify -verbose

"bin\Debug\net8.0-android\com.companyname.autogestionsena.maui-Signed.apk"

7. Compilación con Parámetros Adicionales

Compilación con información detallada (verbose)

dotnet build -f net8.0-android -c Debug -v detailed

Compilación forzando reconstrucción completa

dotnet build -f net8.0-android -c Debug --no-incremental

Compilación especificando versión de Android

```
dotnet build -f net8.0-android -c Debug  
/p:AndroidTargetSdkVersion=34
```

6.3 Instalación de APK en Dispositivo

Esta sección detalla cómo el archivo APK generado se transfiere e instala en el dispositivo móvil (físico o emulador) para que los testers puedan ejecutar y verificar la aplicación.

1. Requisitos Previos del Dispositivo

Para Dispositivo Físico Android

Requisito	Cómo Habilitarlo
Android 5.0+ (API 21+)	Verificar en: Configuración > Acerca del teléfono > Versión de Android
Opciones de Desarrollador	Configuración > Acerca del teléfono → Tocar 7 veces en Número de compilación
Depuración USB	Configuración > Opciones de desarrollador → Activar Depuración USB
Orígenes Desconocidos	Permitir instalación desde fuentes externas (ver sección 2.6)
Espacio disponible	Mínimo 150 MB libres

Pasos para Habilitar Opciones de Desarrollador

- Ir a: Configuración > Acerca del teléfono
- Buscar: "Número de compilación" o "Build number"
- Tocar 7 veces consecutivas
- Mensaje: "¡Ahora eres un desarrollador!"

- Volver a: Configuración > Sistema > Opciones de desarrollador
- Activar: "Depuración USB"
- Conectar el dispositivo por USB al PC
- Aceptar el prompt de depuración en el teléfono (marcar "Siempre permitir")

2. Método 1: Instalación por ADB (Android Debug Bridge) - RECOMENDADO

Este es el método más rápido y confiable para desarrollo y testing.

Prerequisitos

ADB instalado (viene con Android SDK)

Dispositivo conectado por USB con depuración habilitada

Comandos de Instalación

1. Verificar que el dispositivo está conectado y reconocido

adb devices

Salida esperada:

List of devices attached

XXXXXXXXX device

2. Instalar la APK (reemplaza si ya existe)

adb install -r

"C:\Users\braya\Desktop\Front-end-Mobile-Autogestion-Sena\bin\Debug\net8.0-android\com.companynome.autogestionsena.maui-Signed.apk"

3. Instalar con permisos automáticos (Android 6.0+)

adb install -r -g

"bin\Debug\net8.0-android\com.companynome.autogestionsena.maui-Signed.apk"

4. Si hay error de versión/firma, desinstalar primero:

adb uninstall com.companynome.autogestionsena.maui

adb install

"bin\Debug\net8.0-android\com.companyname.autogestionsena.maui-Signed.apk"

Opciones de ADB Install

Opción	Descripción
-r	Reinstalar la app (reemplazar versión existente)
-g	Otorgar todos los permisos automáticamente
-d	Permitir downgrade (instalar versión anterior)
-t	Permitir APKs de test

3. Método 2: Transferencia Manual por USB

Si no tienes ADB configurado o prefieres un método manual:

Paso 1: Copiar APK al dispositivo

Opción A: Usando ADB push

adb push

"bin\Debug\net8.0-android\com.companyname.autogestionsena.maui-Signed.apk" /sdcard/Download/AutoGestion-SENA.apk

Opción B: Usando Windows Explorer

- 1. Conectar el dispositivo por USB
- 2. Seleccionar "Transferir archivos" en el teléfono
- 3. Abrir el dispositivo en Windows Explorer
- 4. Copiar la APK a: Almacenamiento interno > Download

Paso 2: Instalar desde el dispositivo

- Abrir la app "Archivos" o "Gestor de archivos" en el teléfono

- Navegar a la carpeta "Descargas" o "Download"
- Tocar el archivo AutoGestion-SENA.apk
- Si aparece advertencia de fuentes desconocidas, tocar "Configuración"
- Activar "Permitir desde esta fuente"
- Volver y tocar "Instalar"
- Una vez instalada, tocar "Abrir"

4. Método 3: Compartir por Nube o Red

Útil para distribuir a múltiples testers:

Opción A: Google Drive / OneDrive

- 1. Subir la APK a Google Drive
- 2. Obtener enlace de compartir
- 3. Enviar enlace por correo/chat a los testers

Los testers deben:

- Abrir el enlace en el navegador del celular
- Descargar la APK
- Instalar desde Descargas

Opción B: Servidor interno / Red local

Iniciar servidor HTTP simple en Python

```
cd "bin\Debug\net8.0-android"
```

```
python -m http.server 8080
```

Los testers acceden desde el navegador del celular:

#

```
http://[IP_DEL_PC]:8080/com.companyname.autogestionsena.  
maui-Signed.apk
```

Opción C: Envío por correo electrónico

Nota: Gmail y otros proveedores pueden bloquear archivos APK por seguridad. Se recomienda usar un archivo ZIP o servicios de almacenamiento en la nube.

5. Método 4: Instalación en Emulador Android

1. Listar emuladores disponibles

emulator -list-avds

2. Iniciar el emulador

emulator -avd Pixel_5_API_34

3. Esperar a que el emulador inicie completamente

4. Instalar APK en el emulador

adb install

"bin\Debug\net8.0-android\com.companyname.autogestionsena.maui-Signed.apk"

Alternativa: Arrastrar y soltar

- Simplemente arrastrar el archivo APK sobre la ventana del emulador

6. Habilitar "Orígenes Desconocidos" (Android 8.0+)

En Android 8.0 (Oreo) y versiones posteriores, los permisos de instalación se otorgan por aplicación:

Al intentar instalar la APK, aparecerá un mensaje de bloqueo

- Tocar "Configuración" en el mensaje
- Activar "Permitir desde esta fuente" para:
- Chrome (si descargaste desde navegador)
- Archivos (si instalas desde gestor de archivos)
- Gmail (si recibiste por correo)

Volver y completar la instalación

7. Verificación de Instalación Exitosa

Una vez instalada la APK, verificar:

1. Verificar que la app está instalada

adb shell pm list packages | Select-String "autogestionsena"

Salida esperada: package:com.companyname.autogestionsena.maui

2. Obtener información de la app instalada

```
adb shell dumpsys package com.companyname.autogestionsena.maui |  
Select-String "versionName|versionCode"
```

3. Iniciar la aplicación desde ADB

```
adb shell am start -n  
com.companyname.autogestionsena.maui/crc64e1fb321c08285b90.MainActivity
```

- Verificación Manual en el Dispositivo
- Buscar el ícono "AutoGestión SENA" en el menú de aplicaciones
- Abrir la aplicación
- Verificar que carga la pantalla de login
- Probar las funcionalidades básicas
- Verificar conexión con el backend (si aplica)

8. Solución de Errores Comunes de Instalación

Código de Error	Causa	Solución
INSTALL_FAILED_UPDATE_INCOMPATIBLE	La APK tiene firma diferente a la versión instalada	adb uninstall com.companyname.autogestionsena.maui y reinstalar
INSTALL_FAILED_VERSION_DOWNGRADE	Intentas instalar versión anterior	Agregar flag -d: adb install -r -d archivo.apk
INSTALL_PARSE_FAILED_NO_CERTIFICATES	APK no está firmada	Usar el archivo *-Signed.apk en lugar del sin firmar
INSTALL_FAILED_INSUFFICIENT_STORAGE	No hay espacio en el dispositivo	Liberar espacio o usar adb install con almacenamiento externo
INSTALL_FAILED_OLDER_SDK	Versión de Android no compatible	Verificar que el dispositivo tenga Android 5.0+
INSTALL_FAILED	Restricciones de	Verificar políticas

<code>_USER_RESTRICTED</code>	usuario/MDM	de seguridad del dispositivo
-------------------------------	-------------	------------------------------

9. Script Completo de Instalación

```
# =====
# Script: install-apk.ps1
# Propósito: Automatizar la instalación de APK
# =====

param(
    [string]$ApkPath =
"bin\Debug\net8.0-android\com.companyname.autogestionsena.maui-Signed.apk",
    [switch]$Reinstall,
    [switch]$GrantPermissions
)

# Verificar que ADB está disponible
if (-not (Get-Command adb -ErrorAction SilentlyContinue)) {
    Write-Host " ADB no encontrado. Verifica que Android SDK está en el PATH" -ForegroundColor Red
    exit 1
}

# Verificar dispositivo conectado
$devices = adb devices | Select-String "device$"
if (-not $devices) {
    Write-Host " No hay dispositivos conectados" -ForegroundColor Red
    Write-Host "  Conecta un dispositivo con Depuración USB habilitada" -ForegroundColor Yellow
    exit 1
}
```

```
Write-Host " Dispositivo detectado: $($devices -replace '\s+device',)"
-ForegroundColor Green
```

```
# Verificar APK existe
if (-not (Test-Path $ApkPath)) {
    Write-Host " APK no encontrada: $ApkPath" -ForegroundColor Red
    exit 1
}
```

```
$apkInfo = Get-Item $ApkPath
Write-Host " APK: $($apkInfo.Name)
($([math]::Round($apkInfo.Length/1MB, 2)) MB)" -ForegroundColor Cyan
```

```
# Construir comando de instalación
$installCmd = "adb install"
if ($Reinstall) { $installCmd += " -r" }
if ($GrantPermissions) { $installCmd += " -g" }
$installCmd += " ``$ApkPath`""
```

```
Write-Host " Instalando..." -ForegroundColor Yellow
$result = Invoke-Expression $installCmd
```

```
if ($result -match Success) {
    Write-Host "Instalación exitosa!" -ForegroundColor Green
    Write-Host " Iniciando aplicación..." -ForegroundColor Cyan
    adb shell am start -n
com.companyname.autogestionsena.maui/crc64e1fb321c08285b90.MainActivity
} else {
    Write-Host " Error en la instalación:" -ForegroundColor Red
    Write-Host $result -ForegroundColor Yellow
}
```

Uso del script:

Instalación básica

```
.\install-apk.ps1
```

Reinstalar y otorgar permisos

```
.\install-apk.ps1 -Reinstall -GrantPermissions
```

APK específica

```
.\install-apk.ps1 -ApkPath "releases\AutoGestion-SENA-1.0.0-debug.apk"
```

6.4 Compilación del Frontend (Build de Producción)

1. Preparación del Entorno en el Servidor

Antes de clonar el proyecto, se preparó el entorno necesario en el VPS:

- Instalación de Node.js y npm para permitir la construcción del proyecto.
- Creación de la estructura de carpetas dentro de /home/reyving/apps/ destinada a alojar el frontend.
- Verificación del estado del firewall y disponibilidad del puerto 80 para servir la aplicación mediante Nginx.

2. Clonación del Proyecto Frontend

Una vez preparado el entorno, se clonó el repositorio del frontend desde GitHub directamente en el VPS:

- **git clone**
<<https://github.com/July173/Front-end-web-Autogestion-Sena>>
Front-end-web-Autogestion-Sena

Esto permitió obtener la versión más reciente del código fuente para construir y desplegar la aplicación.

3. Generación del Build de Producción

Dentro del proyecto clonado, se instalaron las dependencias y se generó el artefacto final de producción:

- **npm install**
- **npm run build**

El comando `npm run build` generó la carpeta `dist/`, que contiene los archivos optimizados (HTML, CSS, JS) listos para ser servidos por Nginx.

4. Configuración e Instalación del Build en el Servidor Web

El contenido compilado fue transferido al directorio destinado a alojar el frontend público:

- **sudo rm -rf /var/www/frontend/***
- **sudo mv dist/* /var/www/frontend/**
- **sudo chown -R www-data:www-data /var/www/frontend**

Esto garantizó que la aplicación fuera accesible desde el servidor web configurado en el VPS.

5. Integración con el Backend Desplegado Previamente

Para conectar correctamente el frontend con el backend:

- Se configuraron las variables de entorno:
 - `VITE_API_BASE_URL=/api/`
- Nginx se configuró como reverse proxy para enviar todas las solicitudes que comiencen en `/api/` hacia el backend ejecutándose con Gunicorn.

- Se verificó que las rutas del frontend utilizarán la base URL correcta para consumir la API.

La configuración final permitió que el frontend consumiera los servicios REST del backend desde la misma dirección pública, manteniendo una arquitectura unificada y segura.

6. Validación Final del Despliegue

Se realizó una validación completa comprobando:

- Carga del frontend en `http://167.114.98.199/`
- Funcionamiento de rutas internas del SPA
- Comunicación del frontend con el backend por `/api/`
- Correcto retorno de datos desde la API previamente desplegada

Con esto se confirmó que el frontend quedó completamente operativo y correctamente integrado al backend en producción.

6.5 Despliegue del Backend (Producción)

Para el despliegue en producción del backend se utilizó un servidor VPS con sistema operativo Ubuntu, configurando una arquitectura basada en:

- Gunicorn como servidor WSGI para ejecutar la aplicación Django.
- Nginx como servidor web y reverse proxy.
- MySQL como motor de base de datos en ambiente productivo.

Los pasos generales de despliegue fueron:

Preparación del servidor

- Actualización de paquetes del sistema operativo.

- Configuración de firewall con UFW permitiendo únicamente los puertos 22 (SSH), 80 (HTTP) y 443 (HTTPS).
- Creación de la estructura de directorios para el proyecto en /home/reyving/apps/.

Obtención del código fuente

- Clonado del repositorio remoto del backend desde GitHub en la ruta:
- /home/reyving/apps/Back-end-web-API-autogestionSena
- Configuración del entorno virtual de Python:
- python3 -m venv venv
- source venv/bin/activate

Instalación de dependencias

- Instalación de los paquetes de Python definidos en requirements.txt mediante:
- pip install -r requirements.txt
- Instalación de gunicorn como servidor WSGI.

Configuración de la aplicación Django

- Ajuste del archivo settings.py para entorno de producción:
- Configuración de ALLOWED_HOSTS con la IP del servidor.
- Configuración del bloque DATABASES para utilizar MySQL como motor de base de datos productivo.
- Definición de STATIC_ROOT para la recolección de archivos estáticos.
- Configuración de CORS con django-cors-headers para permitir únicamente el dominio/IP del frontend.
- Configuración de CSRF_TRUSTED_ORIGINS para proteger los formularios y autenticación.

Base de datos (MySQL)

- Creación de la base de datos productiva y usuario con permisos específicos para la aplicación.
- Aplicación de migraciones con:
- `python3 manage.py migrate`
 - Opcionalmente, carga de datos iniciales o de un dump .sql actualizado.

Archivos estáticos

- Ejecución de:
- `python3 manage.py collectstatic`
- Nginx se configuró para servir directamente los archivos estáticos desde la carpeta staticfiles.

Configuración de Gunicorn como servicio

- Creación del servicio systemd `gunicorn-autogestion.service`, indicando:
- Ruta del proyecto.
- Ruta del entorno virtual (venv).
- Módulo WSGI del proyecto Django.
- Socket Unix en `/run/gunicorn/gunicorn-autogestion.sock`.
- Habilitación y arranque del servicio:
- `sudo systemctl daemon-reload`
- `sudo systemctl enable gunicorn-autogestion`
- `sudo systemctl start gunicorn-autogestion`

Configuración de Nginx como reverse proxy

- Definición de un bloque server que:

- Escucha en el puerto 80.
- Sirve el frontend estático desde /var/www/frontend.
- Redirige las peticiones a la API (/api/) y Swagger (/swagger/) hacia el socket de Unicorn.
- Sirve los archivos estáticos del backend desde la ruta configurada.
- Prueba de sintaxis: `sudo nginx -t`
- Recarga del servicio: `sudo systemctl reload nginx`.

6.6 Versionamiento

El proyecto utiliza Git como sistema de control de versiones y GitHub como repositorio remoto centralizado, lo que permite llevar un historial completo de cambios, facilitar el trabajo colaborativo y asegurar la trazabilidad de las modificaciones.

Las prácticas de versionamiento implementadas son:

Repositorio remoto en GitHub

- El código fuente del backend y del frontend se mantiene en repositorios separados.
- GitHub almacena la versión “oficial” del proyecto, desde donde se realizan los despliegues a producción.

Ramas principales

- main o master: rama estable que representa la versión desplegada o lista para producción.
- develop: rama de integración donde se acumulan las funcionalidades antes de pasar a producción.
- qa:
- staging:

Flujo de trabajo general

- Se crea una rama a partir de main o develop para cada nueva funcionalidad o corrección.
- Se realizan commits frecuentes, con mensajes descriptivos y claros.
- Una vez finalizada la tarea, se genera un pull request hacia la rama destino (develop o main).
- Se realiza revisión de código (code review) y, tras su aprobación, se hace el merge.
- Para cada versión estable de producción, se puede crear un tag (por ejemplo: v1.0.0, v1.1.0) que identifica un punto concreto del código.

Buenas prácticas de Git

- Mensajes de commit claros y significativos.
- No subir archivos generados (dist, staticfiles, etc.), sino sólo código fuente y archivos de configuración.
- Uso de .gitignore para excluir archivos temporales, entornos virtuales, archivos de configuración local, etc.

6.7 Buenas Prácticas de Despliegue

Para garantizar un despliegue ordenado, reproducible y seguro, se han tenido en cuenta las siguientes buenas prácticas:

Separación de ambientes

- Diferenciar al menos entre:
 - Ambiente de desarrollo (local).
 - Ambiente de producción (VPS).

- Configuración de parámetros sensibles (BD, claves, DEBUG, etc.) mediante variables de entorno o archivos de configuración específicos de cada ambiente.

Gestión de dependencias

- Uso de requirements.txt para controlar la versión de las librerías de Python.
- Instalación de dependencias dentro de un entorno virtual (venv) para evitar conflictos con el sistema operativo.

Despliegue automatizado y ordenado

- Flujo estándar de despliegue:
 - git pull desde la rama estable.
 - pip install -r requirements.txt si hay nuevos paquetes.
 - python3 manage.py migrate para aplicar cambios en la base de datos.
 - python3 manage.py collectstatic para actualizar los archivos estáticos.
 - Reinicio controlado del servicio Gunicorn y recarga de Nginx.
- Evitar cambios manuales directamente en producción que no estén versionados en Git.

Seguridad

- DEBUG = False en producción.
- Restricción de ALLOWED_HOSTS a los dominios y/o IPs válidos.

- Configuración de CORS para permitir únicamente orígenes autorizados (frontend de producción y, opcionalmente, entornos de prueba).
- Protección de credenciales de base de datos y claves secretas (SECRET_KEY) fuera del código fuente (variables de entorno, archivos no versionados).
- Firewall configurado para permitir solo los puertos necesarios (22, 80, 443).

Manejo de base de datos

- Uso de migraciones de Django para evolucionar el esquema de la base de datos.
- Realización de copias de seguridad periódicas (backups) de la base MySQL, especialmente antes de despliegues importantes.
- Posibilidad de restaurar la base de datos a partir de un dump en caso de fallos.

Monitoreo y logs

- Configuración de logs de Gunicorn y Nginx para poder revisar errores y trazas de ejecución.
- Revisión de los logs después de cada despliegue para detectar posibles problemas tempranamente.

Capacidad de rollback

- Al estar el proyecto versionado en Git, es posible regresar a una versión anterior (tag o commit previo) en caso de que una nueva versión presente errores críticos.
- El flujo de despliegue permite revertir rápidamente el código y volver a levantar el servicio con una versión estable.

7. Referencias y Contacto

7.1 Enlaces de Referencia

- **Repositorios GitHub:**

- **Frontend:**
 - <https://github.com/July173/Front-end-web-Autogestion-Sena.git>
- **Backend:**
 - <https://github.com/July173/Back-end-web-API-autogestionSena.git>
- **Móvil:**
 - <https://github.com/July173/Front-end-Mobile-Autogestion-Sena.git>

7.2 Información de Contacto

Desarrolladores:

- **Nombre:** Brayan Stid Cortes Lombana
Ubicación: Neiva, Huila, Colombia
Email: brayanstidcorteslombana@gmail.com
GitHub: brayancortes22
- **Nombre:** Reyving Fabriani Ramirez Medina
Ubicación: Neiva, Huila, Colombia
Email: medinafabriany@gmail.com
GitHub: reivyng
- **Nombre:** July Daniela Ramos Peña
Ubicación: Neiva, Huila, Colombia
Email: july345ra@gmail.com
GitHub: july173